

Input Order Is an Uncontrolled Variable in Lossless Point Cloud Compression Benchmarks

September 23, 2026

Abstract

Lossless point cloud codecs are compared at equal bitrate, but they do not all encode the same object: some reconstruct the multiset of points, others the sequence. The arrangement of the input is not part of any published test conditions, so a reported bitrate is not determined by them. We make the arrangement an explicit factor — eight order conditions on a fixed point set, four lossless coders, 41 blocks from 17 files spanning airborne, automotive and terrestrial scanning, 324 rows — and give the definitions needed to repeat the measurement.

The effect dwarfs the differences the benchmarks are built to resolve, and it changes their verdict. At the file's own order the four coders lie within a 3% of each other and no coder wins everywhere: an octree is shortest on 20 of 41 blocks, a sequential context coder on 12, a scan-model coder on 9. Permuting the same points uniformly costs the sequential coders a median of +115.6% and +130.0%, more than an order of magnitude beyond the spread between coders — and the context coder, shortest on 12 blocks at the file's own order, is shortest on none once the points are permuted. The octree agrees with the original order to three digits in all 324 rows, and still does when its own azimuth sort or predictive-geometry mode is switched on, or when a block exceeds its slice limit.

Measuring this needs two null controls. The seed-to-seed difference between two random permutations has a median of 0.014 percentage points, but reversing the input — which preserves every local neighbourhood — moves one coder by -1.28 to $+1.59$. The larger is the resolution of the experiment, and against it 6 of 32 blocks respond. A coder that derives its coding order from a sort key keeps its invariance until that key degenerates, and the onset is late: of the 27 blocks at up to 62% key duplication only one exceeds the floor, and by a factor of 1.1; a file at 82.4% sits 4.4 times the floor; and fully degenerate keys sit 320 to 495 times it, where the coder collapses onto a plain geometry coder. A controlled sweep that manufactures duplication while holding the input order fixed identifies the cause — restoring injectivity at the same quantisation step returns the cost to zero at every level, however within-group ranks are assigned — but its graded response does not match the threshold seen in real files: manufacturing 60% duplication on an airborne tile takes a quantisation step 2.4 times its firing interval, which merges consecutive shots, while the tile that arrives at 58% keeps its firing interval intact.

Finally we measure the term that a comparison across the two kinds of codec omits. Coding the acquisition-order permutation against the octree's actual output order costs a median of 7.800 bpp, not the 18.489 of the uniform bound; charging it block by block moves the octree's median from 24.45 to 32.63 bpp, and from shortest on 20 of 41 blocks to shortest on none.

1 Introduction

A lossless point cloud codec is judged by bits per point (bpp). The comparison assumes that the coders under test solve the same problem. They do not.

An octree coder such as G-PCC reconstructs the multiset of occupied positions: the order in which the points arrived is discarded. A sequential coder such as LASzip reconstructs the sequence, so that point i of the output is point i of the input. For $n = 10^6$ the difference is

$$\frac{\log_2 n!}{n} \approx \log_2 n - \log_2 e = 18.489 \quad (1)$$

bpp of information that one carries and the other does not, comparable to the entire geometry budget of an airborne LiDAR tile.

What is missing is a measurement. The common test conditions used to compare lossless point cloud coders fix the sequences, the quantisation and the attribute set, but not the arrangement of the points within a file. We treat the arrangement as an experimental factor,

give the definitions needed to reproduce the measurement, and report both the absolute code lengths and their response to order.

1.1 Contributions

The gap in the literature is narrow and we state it narrowly: the measurement. Every component of the order-invariance property is prior art (Section 2).

1. A protocol that makes input order an explicit factor, with two null controls and a stated resolution, defined in Section 3.
2. The measurement it produces: four coders, 41 blocks from 17 files, eight conditions, 324 rows, with absolute code lengths alongside the relative response (Section 6).
3. A quantification of how a key-sorted coder loses order invariance as its key degenerates, in real files and under a controlled sweep that manipulates the

key alone, and of where the two disagree (Section 7).

4. A measured value for a term the literature bounds but, in this setting, does not measure: coding the acquisition-order permutation against the octree’s own output order costs a median of 7.800 bpp, against the 18.489 of the uniform bound (Section 6.3). We make no novelty claim for the coding technique, which is elementary.

2 Related work

Order-invariant geometry coding is standard. The octree of G-PCC and the kd-tree of Draco both code a multiset, so their geometry cost does not depend on the input arrangement. TMC13 additionally offers `--sortInputByAzimuth` and `--predGeomSort`, which re-derive an acquisition-like order internally, whose construction orders include Morton and azimuthal variants. Re-deriving a scan order from the data is prior art. Separately, and for *attribute* coding rather than geometry, the standard carries a `canonical_point_order_flag` that selects between Morton order and decoded-geometry order.

Transmitting the order as a separate channel is disclosed. US10796457B2 (Van Beek, Intel, priority 2018) states that if point positions are coded losslessly the reconstructed X, Y, Z are identical whatever the coding order, and that the choice of coding order may be transmitted so that the original order can be reconstructed. We have not analysed the independent claims and make no statement about claim scope; nor have we obtained the file history. The disclosure is framed around range images and around selecting a coding order from a predefined set, where we code an arbitrary permutation, but we do not claim that distinction carries legal weight.

The $\log n!$ framework is established in the opposite direction. Shuffle coding [2] and Random Order Coding [3] spend effort to *save* the bits of (1) for an unordered set, and the term “canonical ordering” is theirs. The machinery is ours with the sign reversed and we claim none of it. What we add is not a coder but an observation about comparison: that work assumes the order may be discarded, and our measurement shows that published benchmarks disagree silently about whether it may.

LASzip [1] is the representative order-dependent coder: it does not re-sort, and its predictor follows the scan line. Re-sorting is left to external tools, which destroy the input order.

Our own earlier work [4] reports one instance of the effect (one file, 2M points, one direction) and claims order invariance for the attribute columns of a coder whose coding order is derived from the geometry. The present paper measures the geometry columns, across coders and conditions, and identifies the condition under which the invariance fails.

3 Formulation

3.1 Setting

A point cloud is a finite *sequence* $x = (x_1, \dots, x_n)$ with $x_i \in \mathbb{Z}^3 \times \mathcal{A}$, where the integer triple is the quantised coordinate and $\mathcal{A}$ holds the attributes. Write $x_i^g \in \mathbb{Z}^3$ for the coordinate part.

A codec is a pair (E, D) . It is *sequence-lossless* if $D(E(x)) = x$ and *multiset-lossless* if $D(E(x))$ equals x as a multiset. Its code length is reported in bits per *point* (bpp throughout this paper; not bits per pixel),

$$L(x) = \frac{8|E(x)|}{n} \quad [\text{bpp}], \quad (2)$$

where $|E(x)|$ is the coded size in bytes and n is the number of points *presented to the encoder*, counting co-incident points separately. The same n divides all four coders, and none of them merges coincident points: the blocks contain up to 26,661 of them and every round trip reproduces them.

What $|E(x)|$ counts is not identical across coders and we state it rather than elide it. For G-PCC and for LAZ it is the whole coded file, including headers and, for LAZ, a chunk table; on the smallest block that order-independent fixed part is about 0.3 bpp. For our two coders it is the geometry substream alone, without the container that carries the schema and the checksums. A fixed additive term compresses the relative change (3), so the LAZ column’s Δ is slightly conservative relative to ours; at 10^6 points the fixed part is below 0.01 bpp and the effect is negligible, but at 10^4 it is not.

For $\pi \in S_n$ let $x \circ \pi = (x_{\pi(1)}, \dots, x_{\pi(n)})$. The quantity we measure is the relative change of code length under a rearrangement,

$$\Delta(\pi) = 100 \left(\frac{L(x \circ \pi)}{L(x \circ \text{id})} - 1 \right) \quad [\%], \quad (3)$$

with the file’s own order as the baseline. A multiset-lossless codec is not *required* by its definition to have $\Delta \equiv 0$; whether it does is an empirical question about the implementation, and we measure it rather than assume it.

3.2 Order conditions

Eight conditions, all on the same point set.

Identity. $\pi = \text{id}$, the order in the source file.

Reverse. $\pi(i) = n + 1 - i$.

Morton. π sorts the points by the Morton code of their coordinates. Each axis is first normalised to its own range, which matters because raw coordinates can exceed the width of the interleaved word:

$$b_{i,c} = x_{i,c}^g - \min_j x_{j,c}^g, \quad r_c = \max_j \left(\max_j b_{j,c}, 1 \right), \quad (4)$$

$$a_{i,c} = \left\lfloor b_{i,c} (2^{21} - 1) / r_c \right\rfloor, \quad (5)$$

and $m(i) = \sum_{c=0}^2 \sum_{k=0}^{20} ((a_{i,c} \gg k) \& 1) \ll (3k + c)$ is the interleaving. The sort is stable.

Window w . Independent uniform permutations inside consecutive blocks of w indices: π restricted to

$\{jw + 1, \dots, \min((j+1)w, n)\}$ is uniform on that block, independently for each j . It interpolates between the identity ($w = 1$) and a uniform permutation ($w = n$). We use $w \in \{10^2, 10^3, 10^4\}$ and apply a condition only when $10w \leq n$, so that a window is never a substantial fraction of the block.

Uniform. $\pi \sim \text{Uniform}(S_n)$, drawn twice. The two draws are consecutive outputs of one PCG64 stream seeded with a fixed constant, restarted for each block, so they are independent of each other but not of the block; the window conditions draw from the same stream first.

3.3 Resolution

A difference in Δ is only meaningful above the variation the apparatus produces on its own. We use two controls.

The *draw control* is the difference between the two uniform draws,

$$\sigma = \max_{\text{blocks}} |\Delta(\pi_1) - \Delta(\pi_2)|, \quad (6)$$

which measures variation between two draws of one treatment, with the baseline cancelling. It is not a re-seeding: the stream is the same.

The *reverse control* is

$$\rho_{\text{rev}} = \max_{\text{blocks}} |\Delta(\text{rev})|. \quad (7)$$

Reversal preserves every local neighbourhood, so a direction-symmetric sequential predictor should be unaffected; the deviation measures what a benign rearrangement costs. The *effective floor* is

$$\varphi = \max(\sigma, \rho_{\text{rev}}), \quad (8)$$

computed per coder. We call an observed $|\Delta|$ resolved only if it exceeds φ .

3.4 Key duplication

One of the coders sorts internally by a key $k_i = (s_i, t_i)$, the flight line identifier and the timestamp. Its degeneracy is

$$\rho_{\text{key}} = 1 - \frac{|\{k_1, \dots, k_n\}|}{n} \in [0, 1), \quad (9)$$

the fraction of points whose key is shared with an earlier one.

Proposition 1 *If $\rho_{\text{key}} = 0$ and the coder's internal sort is a stable sort on k , then $L(x \circ \pi) = L(x)$ for every $\pi \in S_n$.*

With $\rho_{\text{key}} = 0$ the key is injective, so it induces a total order; a stable sort returns that order whatever the input arrangement, and the encoder sees the same sequence. Stability is irrelevant here but becomes the whole question when $\rho_{\text{key}} > 0$, because ties are then broken by input position. Sections 6 and 7 measure what happens in that regime.

4 The four coders

All four are measured on geometry alone, so that the levels are comparable.

G-PCC. TMC13 `release-v23.0-rc2`, lossless octree, geometry only, driven through a PLY file with `float32` coordinates, which bounds the integer range the harness can carry. Multiset-lossless.

LAZ. LASzip through a LAS 1.4 point record of type 6 containing only X, Y, Z . The figure is the whole file divided by n , so it includes a header and a chunk table that do not depend on the order; on the smallest block this fixed part is about 0.3 bpp. Sequence-lossless.

v3, our sequential context coder. Points are coded in input order; for each point the three axes are coded in turn. Let $d_{i,c} = x_{i,c}^g - x_{i-1,c}^g$ and let the predictor be the running median of the last three differences,

$$\hat{x}_{i,c} = x_{i-1,c}^g + \text{med}(d_{i-1,c}, d_{i-2,c}, d_{i-3,c}). \quad (10)$$

The recursion starts from $x_0^g = 0$, so the first three points use a shorter window and the first stored difference is x_1^g itself; this affects a handful of points in 10^6 but changes the bitstream, and the container is checked for byte equality across runs. The residual is mapped to a non-negative integer by $\zeta(v) = 2v$ for $v \geq 0$ and $-2v - 1$ otherwise, and $z_{i,c} = \zeta(x_{i,c}^g - \hat{x}_{i,c})$ is written by an adaptive range coder with a context selected by magnitude. With $\kappa(z) = \min(\lceil \log_2(z + 1) \rceil, 23)$ giving 24 buckets per axis, the context index is

$$\gamma_{i,c} = \begin{cases} \kappa(z_{i-1,0}) & c = 0, \\ \kappa(z_{i,0}) & c = 1, \\ \lfloor (\kappa(z_{i,0}) + \kappa(z_{i,1})) / 2 \rfloor & c = 2. \end{cases} \quad (11)$$

Axis 0 takes its context from the previous point; axes 1 and 2 take theirs from the axes already coded at the same point. This cross-axis coupling is what distinguishes v3 from a plain first-difference coder.

scan, our scan-model coder. It sorts the points by k_i with a stable sort, splits the sorted sequence into sweeps wherever the timestamp gap exceeds 20 times the median firing interval, and fits a model per sweep. Within a sweep, the horizontal coordinates are rotated into the sweep plane by three integer shears, each a bijection of $\mathbb{Z}^2$, so the rotation is exactly invertible without floating point. Writing s for the in-plane coordinate, z for height, S_z for the sensor height, θ_0 for the scan angle at the start of the sweep and ω for the angular step per unit of j , where j is the timestamp measured as an integer offset in units of the last place of the `float64` representation, the prediction is

$$\hat{s}_j = s_0 + \left\lfloor (S_z - z_j) \tan(\theta_0 + \omega j) \right\rfloor, \quad (12)$$

with the angle clamped to $\pm 45^\circ$ and the tangent read from a fixed-point table built by an integer CORDIC, so the decoder never evaluates a transcendental function. Sweeps are cut wherever the flight line changes or the timestamp gap exceeds twenty times the median positive gap. The five parameters $(s_0, S_z, \theta_0, \omega)$ are fitted by Levenberg–Marquardt per scan line, the

lines being separated by an expectation-maximisation split; the shear constants come from a principal-axis estimate and the out-of-plane reference from a median. All seven, plus a per-line adoption flag, are written as side information. A line whose fit does not pay for itself, judged on a bit-length proxy rather than the coded size, falls back to (10) applied to the sheared in-plane coordinate. Equation (12) predicts the in-plane coordinate only; the out-of-plane residual uses (10) and the height a per-return-number predictor. Residuals are coded by the same range coder as v3 but with per-axis contexts only — the cross-axis coupling of (11) is not used here. Sequence-lossless.

4.1 Reproduction

The measurement is reproducible only with the exact invocations, which prose cannot carry; we give them here and in the accompanying scripts. It also depends on the build of our container: the numbers in this paper were measured with builds up to commit 77d3e77 (September 22, 2026), when they were last brought up to date. Later changes to the coder, from September 23 on (among them how the binary models adapt their probabilities), alter the bit string of every stream, so a current build gives different code lengths for the same invocations. G-PCC runs with `--trisoupNodeSizeLog2=0` `--mergeDuplicatedPoints=0` `--inferredDirectCodingMode=1` `--neighbourAvailBoundaryLog2=8` `--intra_pred_max_node_size_log2=6` `--planarEnabled=1` `--maxNumQtBtBefore0t=4` `--minQtbtSizeLog2=0`, on coordinates translated to a non-negative octant. Our container runs with the geometry candidate pinned and the fallback disabled. Pinning also requires disabling the symbol-coded variant (PCC_FSYM_CAND=0): by default the container additionally tries a variant that codes the low bits of each residual as a single symbol, and that variant, not the pinned one, is then the configuration it reports. The range coder is binary and adaptive, with a unary prefix and a per-position model, 24 magnitude buckets per axis and no sharing between axes, never reset within a stream. The permutation of Section 6.3 is coded as LEB128 variants over zigzag-mapped first differences with a leading zero, then LZMA at preset 9 with the extreme flag. Coordinates are quantised to 1 mm only for the sources that arrive as floating point; LAS files keep their own header scale, so the quantisation step is not uniform across the corpus.

4.2 What is compared

Table 1 separates what the decoder must reconstruct from whether the code length depends on the arrangement. A coder in the top row is not comparable at equal bitrate with one in the bottom row, because the bottom row also transmits the permutation, worth (1).

Table 1: Lossless coders divide on two independent axes. A comparison at equal bitrate is meaningful only within a row.

	Cost depends on order	Cost independent
Multiset	—	G-PCC (single slice)
Sequence	LAZ, v3	scan, conditionally

Table 2: Geometry code length at the file’s own order, in bpp, over the 41 blocks. The scan coder is defined on 32 of them. “best” counts the blocks on which each coder is shortest among those defined there.

	<i>n</i>	median	min	max	best
G-PCC	41	24.451	2.775	28.081	20
LAZ	41	24.561	6.806	36.615	0
v3	41	24.505	6.698	37.025	12
scan	32	23.800	6.279	36.920	9

5 Experimental setup

Seventeen files. Airborne LiDAR tiles in the LAS/LAZ family (AHN3, AHN4 $\times 2$, AHN5, Autzen $\times 2$, Red Rocks, a survey block carrying extra bytes, and two USGS 3DEPTiles from Alaska and New York), five small LAS test files, some synthetic, one automotive sequence (KITTI Velodyne frames concatenated in acquisition order) and two terrestrial laser scans of a lecture hall. Twelve large files contribute three disjoint blocks of 10^6 points each, below the default `sliceMaxPoints` of TMC13 so that G-PCC codes a single slice; five small files are used whole (10,683 to 110,000 points). This gives 41 blocks and $41 \times 8 - 4 = 324$ rows once the four window conditions that the small blocks cannot support are removed. The four coders are columns.

The automotive and terrestrial files carry no per-point timestamp and no flight line, so the scan coder’s key cannot be formed. We leave its column empty on those nine blocks rather than synthesise a key, which would manufacture the very quantity Section 7 is about. Their floating-point coordinates are quantised to 1 mm; no coder merges coincident points, of which the blocks contain up to 26,661.

Every row is decoded and compared. For our own coder the candidate is pinned, so the configuration reported is the configuration verified, and the fallback that would substitute an embedded container is disabled. “Verified” means equality of all columns in order for LAZ and for our container, and equality of the multiset of coordinates after a lexicographic sort for G-PCC. All 324 rows pass, with no missing conditions and no duplicated rows.

6 Results

6.1 Absolute code length

Table 2 gives the levels the benchmark is trying to resolve. The four medians span 0.76 bpp, about 3%; block by block the median difference from G-PCC is +4.6% for LAZ, +0.3% for v3 and +7.0% for scan, and each sequential coder is shorter than G-PCC on some blocks

Table 3: The same comparison under each order condition. Median code length in bpp (the scan column over the 32 blocks where it is defined, the others over 41; the window 10^4 row over 37 and 28), and the number of blocks on which each coder is shortest, among those defined on that block.

Condition	G-PCC	LAZ	v3	scan	shortest
identity	24.45	24.56	24.50	23.80	20/0/12/9
reverse	24.45	24.57	24.51	23.80	20/0/12/9
Morton	24.45	28.36	29.15	24.13	29/0/0/12
window 10^2	24.45	38.37	38.72	24.64	29/0/0/12
window 10^3	24.45	42.05	42.92	25.01	29/0/0/12
window 10^4	24.59	44.88	46.00	24.77	26/0/0/11
uniform	24.45	49.81	51.62	24.98	29/0/0/12

and longer on others (13 of 41, 19 of 41 and 12 of 32 respectively). LAZ is never the shortest of the four, because v3 differs from it only by the cross-axis context of (11), which does not hurt on this data.

The extremes are wide in both directions. The scan coder is 30.9% shorter than G-PCC on one airborne block (17.098 against 24.755 bpp) and 126.3% longer on a small synthetic plane that the octree codes in 2.775 bpp. No coder dominates.

Cost has to be measured on the same job. The figures above come from a container that also codes 22 attribute columns the octree never sees, and from a harness that forked the timer from a process already holding the point data, so the child’s resident set included its parent’s. Measured on geometry alone, from a lean helper, encoding 10^6 points in the file’s own order takes 1.86 s and 161 MB for G-PCC against 0.43 s and 53 MB for v3 and 0.55 s and 146 MB for the scan coder. The advantage in time is about fourfold and holds to 4×10^6 points (1.77 s for v3 and 2.26 s for the scan coder against 7.89 s).

Memory divides the two sequential coders. The octree slices its input, so its working set is bounded (102, 161, 147 and 266 MB at 5×10^5 , 10^6 , 2×10^6 and 4×10^6 points). Ours grows with the input, but v3 stays below the octree at every size we measured (30, 53, 92 and 206 MB); the scan coder is below it to 10^6 points and above beyond that (77, 146, 285 and 592 MB), because it holds the scan order and the residuals of all three axes at once. Columns are held at the width each one needs rather than at 64 bits, and the neighbour table is built only when some column uses it.

6.2 The verdict depends on the order

Table 3 repeats the comparison under each condition. The coder a benchmark would declare best changes with the arrangement of the input, and one coder disappears entirely: v3 is shortest on 12 of 41 blocks at the file’s own order and on *none* once the points are permuted, at any window width.

This is the practical form of the problem. A comparison run on files in acquisition order and a comparison run on the same files after a sort return different rankings, and nothing in the test conditions says which was used.

Table 4: Code length relative to the file’s own order, per cent, by (3). Median over 41 blocks (32 for scan; window 10^4 over 37 and 28). The last row gives the interquartile range of the uniform condition.

Condition	G-PCC	LAZ	v3	scan
reverse	+0.0	−0.0	−0.0	+0.0
Morton	+0.0	+13.4	+20.7	+0.0
window 10^2	+0.0	+58.5	+63.5	+0.1
window 10^3	+0.0	+76.1	+82.6	+0.1
window 10^4	+0.0	+88.0	+94.1	+0.0
uniform 1	+0.0	+115.6	+130.0	+0.0
uniform 2	+0.0	+115.6	+130.0	+0.1
IQR, uniform	—	[89, 139]	[98, 151]	[−0.1, 1.2]

6.3 Charging the permutation

The octree column of Table 3 does not move: its median is 24.45 bpp under every condition. That constancy is not robustness. It is the signature of a coder that is not carrying the quantity being varied, and it is the reason Table 1 forbids comparing across its rows.

We measured the omitted term rather than bounding it. The reference order is the octree’s actual decode output order, recovered from the reconstructed cloud that the decoder already writes; Morton order is a poor proxy, agreeing with it on none of the positions. Coding the rank sequence of the acquisition order by zigzag-mapped first differences, variable-length integers and LZMA costs a median of 7.800 bpp over the 41 blocks; block by block that is a median of 45.9% of each block’s own bound (1), which is 18.489 for the 37 blocks of 10^6 points and between 11.94 and 15.31 for the four smaller ones. The same coder spends a median of 22.874 bpp on a uniform permutation, 23.7% above the 18.489 optimal there, so it is not an efficient permutation coder and the true cost is lower still; 7.800 is an upper bound that a coder actually attains, not a limit.

Charging it block by block moves the octree’s median from 24.45 to 32.63 bpp. At the file’s own order it is then shortest on *none* of the 41 blocks, against 23 for v3, 15 for scan (of the 32 on which scan is defined) and 3 for LAZ. Under a uniform permutation it recovers, winning 14 of 41: nine of those carry no sort key at all, so only the sequential coders compete there; four are the fully degenerate blocks where the scan coder has lost its own invariance (Section 7); the fourteenth is *vegetation*, a small block whose octree code is short in absolute terms. The octree’s apparent advantage at the file’s own order is an artefact of not charging it for the order.

6.4 Order sensitivity

A uniform permutation of the same points costs LASzip a median of 115.6% and v3 130.0% of their original-order geometry budget, with extremes of +36.0% and +828.6%. Set against Table 2, where the four medians span 3%, the arrangement of the input matters more than the choice of coder by more than an order of magnitude.

G-PCC agrees with the original order to three signif-

Table 5: The two null controls of Section 3.3, in percentage points, and the effective floor (8). Over 41 blocks, or 32 for the scan coder.

	σ med.	σ	rev. min	rev. max	φ
G-PCC	0.000	0.000	+0.000	+0.000	0.000
LAZ	0.008	0.664	-0.385	+0.588	0.664
v3	0.013	0.261	-1.099	+0.070	1.099
scan	0.014	0.305	-1.282	+1.591	1.591

icant digits in every one of the 324 rows, on airborne, automotive and terrestrial data alike. At 10^6 points three digits is a resolution of 0.001 bpp, or 125 bytes, so the statement is “agrees to three digits”, not “exactly zero”.

We also tried to break that invariance and could not. Enabling the encoder’s own `--sortInputByAzimuth`, and separately its predictive-geometry mode with a Morton construction order, leaves the maximum variation across conditions at 0.000% in both cases, over 324 rows each. Doubling the block to 2×10^6 points, above `sliceMaxPoints`, also leaves it at 0.000% over 120 rows from 15 blocks of five airborne files: with the default `UniformSquare` partitioning, slices are cut by position, not by arrival order.

6.5 Resolution

The two controls differ by a factor of five for the scan coder (Table 5). Ten of its 32 blocks move further under reversal than the largest seed-to-seed difference, so a floor built from the seed control alone would count as responses differences that a neighbourhood-preserving rearrangement also produces. All results below are stated against φ .

6.6 Window width and Morton order

Code length increases monotonically in the window width for LAZ (37 of the 37 blocks that support all four conditions) and for v3 (37 of 37). For the scan coder it is monotone on 3 of 28, and those three are the blocks of the one fully degenerate file large enough to enter the count, where the response is the largest effect in the study ($+225 \rightarrow +317 \rightarrow +413 \rightarrow +522\%$). The rest are flat at the level of the floor.

Sorting into Morton order raises the median cost by 13.4% for LAZ and 20.7% for v3, but the sign is not uniform: Morton order shortens the code on 9 of 41 blocks for LAZ, 8 of 41 for v3 and 14 of 32 for scan, with a range for v3 of -21.0% to $+232.3\%$. “A space-filling curve is bad for sequential predictors” is a statement about a median, not a rule.

7 Order invariance and key degeneracy

Table 6 measures the regime the proposition does not cover. Against the floor $\varphi = 1.591$:

Table 6: Cost of a uniform permutation by file, against key duplication (9). The representative block of a file is the one whose duplication is the median, and its figure is the mean of the two uniform draws. The last three files carry no key, so the scan coder has no column there.

File	ρ_{key}	LAZ	v3	scan
red-rocks	1.000	+458	+513	+517
simple1_4	1.000	+829	+642	+787
plane	0.824	+189	+209	+7.0
USGS NY	0.608	+60	+66	-0.7
workshop	0.392	+133	+146	+1.8
fullwave	0.337	+69	+86	+0.5
AHN3	0.240	+123	+134	+0.7
AHN4 (b)	0.183	+136	+149	-1.1
AHN5	0.155	+116	+131	+0.2
USGS AK	0.130	+100	+102	-0.1
autzen-trim	0.097	+119	+128	-0.4
AHN4 (a)	0.083	+124	+135	-0.2
autzen-2023	0.073	+84	+87	+0.0
vegetation	0.000	+56	+56	+0.0
KITTI	—	+145	+165	—
TLS 1	—	+117	+136	—
TLS 2	—	+116	+131	—

- $\rho_{\text{key}} \leq 0.62$ (27 blocks): unresolved, with one exception at 1.1φ ;
- $\rho_{\text{key}} = 0.824$ (1 file): 4.4φ , and already saturated at the narrowest window ($+6.86$ at $w = 10^2$, $+7.06$ at 10^3 , $+7.01$ under a uniform permutation; $w = 10^4$ does not fit the block);
- $\rho_{\text{key}} = 1$ (4 blocks): 320 to 495φ , and graded across the window conditions.

Six of the 32 blocks with a key exceed the floor. At full degeneracy the coder loses the property outright: on red-rocks the scan coder ($+517\%$) lands on the plain geometry coder ($+513\%$), having collapsed into one.

The onset is late. An earlier version of this experiment had no data between $\rho_{\text{key}} = 0.45$ and 0.824 and could not say where the property fails. A USGS tile falls in that gap at 0.577 to 0.624 across its three blocks, and its cost of a permutation is -0.6 to -0.8% , below the floor. The transition therefore lies between 0.62 and 0.824 , and what precedes it is flat rather than rising.

Group size is the obvious alternative explanatory variable — the USGS tile has a median tie group of 2 against 8 for **plane** — but it does not order the effects better: over the 32 blocks with a key, the rank correlation with the cost of a permutation is $+0.457$ for the duplication rate, $+0.457$ for the mean group size and $+0.303$ for the median group size.

The correlation carries no weight. Over the 14 files with a key the rank correlation between ρ_{key} and the cost of a permutation is $+0.607$ ($p = 0.021$), but there are nine such tests in this paper and the Bonferroni bound is 0.19 . Excluding the two fully degenerate files it falls to $+0.378$ ($p = 0.23$). It is also unstable in both directions: of the twelve single exclusions, eleven leave it insignificant and one — dropping the USGS tile, the only file in the gap — raises it to $+0.655$

Table 7: Input order held at the identity; key duplication manufactured by quantising the timestamp. Cost of a uniform permutation, per cent. “rev” is the reverse-order control at the same quantisation step; “dith” restores injectivity at the same step, with within-group ranks taken in storage order and at random.

ρ_{key}	vegetation				AHN4			
	quant	rev	dith		quant	rev	dith	
			sto	rnd			sto	rnd
0.10	-1.0	-2.0	0.0	0.0	+2.1	+4.3	0.0	0.0
0.25	-1.8	-3.9	0.0	0.0	+7.4	+12.1	0.0	0.0
0.50	-0.8	-4.9	0.0	0.0	+14.2	+15.2	0.0	0.0
0.60	+0.6	-4.0	0.0	0.0	+17.3	+15.7	0.0	0.0
0.70	+4.0	-2.6	0.0	0.0	+19.9	+13.8	0.0	0.0
0.80	+8.7	-1.9	0.0	0.0	+24.7	+12.0	0.0	0.0
0.90	+16.3	-0.6	0.0	0.0	+31.3	+8.3	0.0	0.0
1.00	+19.7	-0.0	0.0	0.0	+125.9	-0.1	0.0	0.0

($p = 0.029$). A coefficient that a single file can move across the threshold either way supports nothing. The claims above rest on the four levels of the table.

7.1 A controlled sweep

Table 6 is observational: key duplication is confounded with everything else that distinguishes one survey from another. To break the confound we hold the input order at the identity and manufacture duplication by quantising the timestamp, on two files, in Table 7.

The dither identifies the cause. At every level, restoring injectivity at the same quantisation step returns the cost of a permutation to zero at the 0.1-point resolution of our report. It does so whether the within-group ranks are assigned in storage order or at random, so the effect is caused by degeneracy of the key and not by the loss of timestamp resolution that produced it, nor by any order information the dither might smuggle back in. The control coder, v3, is unaffected throughout: 20.984 bpp in all three arms and at all levels on AHN4, and 25.592 on vegetation.

The sweep and the real files disagree about the shape. Read against the reverse control at the same step — which is not negligible here, reaching 15.7 points on AHN4, because quantising the timestamp changes what the scan model fits — the excess grows monotonically under the sweep: from +1.0 to +19.7 points on vegetation and from -2.2 to +126.0 on AHN4. Table 6 instead shows a threshold, with nothing at $\rho_{\text{key}} = 0.61$ in a real file.

The two are not the same treatment, and the firing interval says why. Manufacturing duplication by quantising a timestamp destroys resolution as well as uniqueness, and the amount destroyed is what matters. On AHN4 the median interval between consecutive shots is 6.0×10^{-7} s, while the step needed for 60% duplication is 1.4×10^{-6} s — 2.4 times the firing interval, so consecutive shots are merged and the scan model loses the covariate it fits. That is what the large reverse control reflects. On vegetation the firing interval is 0.91 s and no step below full degeneracy comes near it, which is why its reverse control stays under five points. The

USGS tile has 57.7% duplication with its firing interval intact at 8.3×10^{-6} s: its ties are between returns of the same shot, not between shots.

So the sweep establishes the cause — degeneracy, not resolution, since the dither removes the effect at a fixed step — while the magnitude it reports at intermediate levels is inflated by the resolution it also destroys. It does not establish the shape of the response, and we do not claim one.

What the sweep cannot be used for. Only the Δ column. The absolute code length at the identity order depends on how within-group ranks are assigned: at $\rho_{\text{key}} = 1$ the scan coder reads 27.079 against 38.549 bpp on vegetation and 23.156 against 50.252 on AHN4 for the two dither arms. Levels are not comparable across arms or across levels.

8 Implications for benchmarks

The order must be part of the test conditions. A lossless bitrate reported without an order condition is not determined. A uniform permutation, the worst case, costs a sequential coder a median of +115.6% (interquartile range +89 to +139%, extremes +36.0 and +828.6%). The rearrangement that actually occurs in practice is milder and its sign is not fixed: sorting an acquisition-order file into a space-filling-curve order, which is what the standard tools do, costs a median of +13.4% and ranges from -21.4% to +304.1%.

Reordering the input is not a levelling operation. Sorting every input into a canonical order does not neutralise an advantage; it destroys the premise of the order-dependent coders, and the canonical order is not neutral either.

Compare within a row of Table 1, or pay the difference. If a multiset coder is compared with a sequence coder, the permutation has to be accounted for on one side. The bound (1) is loose: measured against the octree’s own output order the permutation costs 7.800 bpp (Section 6.3), and charging that much moves the octree from being shortest on 20 of 41 blocks to being shortest on none. A table that lists the two kinds of coder together without an entry for this term is comparing two different quantities.

The multiset side is robust, and that is worth recording. We tried to break G-PCC’s invariance and could not: not with its own azimuth sort, not in predictive-geometry mode, and not above its slice limit, where **UniformSquare** partitioning cuts slices by position rather than arrival. An order clause may treat multiset coders as invariant, provided it says this was checked rather than assumed.

A minimum specification. An order clause needs to fix: (i) which arrangement is the baseline; (ii) whether coders may re-sort internally and whether the original order must be recoverable; (iii) how the permutation is accounted for when the two sides differ on (ii); (iv) null controls with a stated resolution — two of them, since a benign rearrangement can move a coder five times as far as a reseeding does; and (v) publication of the per-block values, not only summaries, since

the medians in Table 4 conceal interquartile ranges of 50 percentage points.

9 Limitations

Sample size. Seventeen files, of which four airborne tiles share a footprint, two share a site and two are scans of the same room: about eleven independent sites. Nothing in this paper rests on the correlational evidence; the statements in Section 7 are made against the floor, block by block.

Confounding. In the observational data, key duplication is confounded with survey identity. The controlled sweep breaks the confound but changes timestamp resolution along with uniqueness, so it identifies the cause without settling the shape of the response.

The gap in the key axis. The observational data now contain a file at $\rho_{\text{key}} = 0.61$ but nothing between 0.62 and 0.824, so the location of the transition is bracketed, not resolved.

Accounting asymmetry. The scan coder’s figure covers X, Y, Z only, and those columns cannot be decoded without the timestamp and flight line that form the key. Including the side columns raises the cost of a uniform permutation to +137.9% in a separate pilot measurement whose per-candidate values were not round-trip verified; it is the only estimate available. The correct statement is conditional: given the side columns, the geometry cost of the scan coder is order-independent at the duplication rates where we can resolve it, $\rho_{\text{key}} \leq 0.62$.

Scope. No sequence from the MPEG common test conditions, and the automotive and terrestrial data carry no key, so the scan coder is measured on airborne data only. Two of the four coders are our own and unpublished; the headline figure for the sequence row, LASzip at +115.6%, does not depend on them.

Novelty. Every component of the invariance property is prior art (Section 2). What is new is the measurement, the protocol, the bracketing of the degeneracy regime, and the measured cost of the omitted permutation term — for which we have not carried out a prior-art search as careful as the one behind Section 2, since the literature on coding permutations is large.

10 Conclusion

Input order moves the code length of sequential lossless point cloud coders by a median of +115.6% and +130.0% under a uniform permutation, more than an order of magnitude beyond the 3% that separates the four coders at the file’s own order. It is large enough to decide the comparison: a coder shortest on 12 of 41 blocks in acquisition order is shortest on none once the points are permuted. It is absent from the test conditions used to compare lossless point cloud coders, and it interacts with coder internals in ways that are not monotone in sign. Making it explicit costs one axis in the experimental design and two null controls; leaving it implicit means the reported ranking is a property of how the test files happened to be written.

References

- [1] M. Isenburg, “LASzip: lossless compression of LiDAR data,” *Photogrammetric Engineering and Remote Sensing*, 79(2), 2013.
- [2] J. Kunze et al., “Entropy coding of unordered data structures,” *ICLR*, 2024.
- [3] D. Severo et al., “Compressing multisets with large alphabets,” *IEEE Journal on Selected Areas in Information Theory*, 2022.
- [4] Lossless coding of airborne LiDAR attributes in a geometry-derived order. Unpublished manuscript, 2026.